

py2: Eine Syntaxerweiterung für Python

Kognitionspsychologische Grundlagen mathematischer
Notation in numerischen Algorithmen

Stephan Epp

9. April 2026

Inhaltsverzeichnis

1	Einführung	2
1.1	Problemstellung	2
1.2	Motivation	2
2	Grundlagen	3
2.1	Definitionen	3
2.2	Grundidee	3
2.2.1	Array2D – die Kernklasse	3
2.2.2	Der py2-Präprozessor	4
2.2.3	Das Leitbeispiel: Crout-LU-Zerlegung	4
3	Kognitionspsychologische Grundlagen	4
3.1	Arbeitsgedächtnis und Miller’s Law	4
3.2	Mental Models und Repräsentationskongruenz	5
3.3	Perzeptuelle Transparenz	5
3.4	Kognitive Dissonanz und Notation	6
3.5	Flow-Theorie und Implementierungsqualität	6
4	Technische Architektur	6
4.1	Projektstruktur	6
4.2	Die Klasse Array2D	7
4.3	Die Klasse Array1D	7
4.4	Arithmetik und Operatoren	8
4.5	Der Präprozessor	8
4.5.1	Übersicht	8
4.5.2	Reguläre Ausdrücke	8
4.5.3	Transformation einer Zeile	9
4.6	Kommandozeilenschnittstelle	10
5	Das Leitbeispiel: Crout-LU-Zerlegung	10
5.1	Mathematische Grundlagen	10
5.1.1	Zerlegungsphase	10
5.1.2	Rückwärtssubstitution	10

5.2	Implementierung in py2	10
5.3	Generierter Python-Code	11
6	Formale Analyse	12
6.1	Korrektheit des Präprozessors	12
6.2	Array-Erkennung: Vollständigkeit und Korrektheit	12
6.3	Komplexität des Präprozessors	13
6.4	Grenzen des Ansatzes	13
7	Psychologische Bewertung der Designentscheidungen	13
7.1	Warum runde Klammern?	13
7.2	Warum keine Tupel?	14
7.3	Warum ein Präprozessor statt Operator-Overloading?	14
7.4	Kognitive Last: Vergleich der Varianten	14
8	Erweiterungsmöglichkeiten	15
8.1	Erweiterung auf n -dimensionale Arrays	15
8.2	Typisierung und statische Analyse	15
8.3	Integration eines Compilers	15
8.4	REPL-Integration	15
9	Zusammenfassung	15
9.1	Kognitionspsychologische Schlussfolgerung	16
9.2	Ausblick	16
9.3	Implementierungshinweis	16

1 Einführung

Das py2-Projekt ist eine domänenspezifische Präprozessor-Pipeline für Python, die eine intuitive Klammer-Syntax für zweidimensionale Array-Zuweisungen der Form $\mathbf{a}(i, j) = \mathbf{expr}$ ermöglicht. Die vorliegende Arbeit analysiert dieses Projekt aus kognitionspsychologischer Perspektive und belegt, dass die Syntaxerweiterung keine ästhetische Spielerei ist, sondern eine wissenschaftlich begründete Maßnahme zur Reduktion extrinsischer kognitiver Last. Zentrale Konzepte – Arbeitsgedächtniskapazität, Chunking, Mental Models, perzeptuelle Transparenz und Notationspsychologie – werden eingehend diskutiert und mit der konkreten Implementierung verknüpft. Darüber hinaus werden Korrektheit, Grenzen und Erweiterungsmöglichkeiten des Präprozessors formal untersucht.

Das py2-Projekt stellt eine kleine Präprozessor-Pipeline bereit, die eine eigene Syntax (.py2) in ausführbares Python 3 übersetzt. Das zentrale Merkmal ist die Möglichkeit, 2D-Array-Zuweisungen in der Form

Listing 1: py2-Syntax: mathematisch-natürliche Schreibweise

```
1  $\mathbf{a}(i, j) = \mathbf{a}(i, j) - \mathbf{a}(i, k) * \mathbf{a}(k, j)$ 
```

zu schreiben, anstatt des umständlichen nativen Python-Äquivalents

Listing 2: Natives Python: strukturell fremd für Mathematiker

```
1  $\mathbf{a}.__setitem__((i, j), \mathbf{a}[i][j] - \mathbf{a}[i][k] * \mathbf{a}[k][j])$ 
```

Der Präprozessor übersetzt jede .py2-Datei vollautomatisch in valides Python 3 und macht damit die Kluft zwischen mathematischer Notation und Programmiersprache unsichtbar.

1.1 Problemstellung

In numerischen Algorithmen – LU-Zerlegungen, finite Differenzenschemata, iterative Löser – ist die Schreibweise a_{ij} oder $a(i, j)$ seit dem 18. Jahrhundert universell etabliert. Python hingegen verwendet eckige Klammern und mehrstufige Indizierung: $\mathbf{a}[i][j]$. Diese Diskrepanz erzeugt bei jedem Lese- und Schreibvorgang eine kognitive Übersetzungsleistung, die die Arbeitsgedächtniskapazität des Programmierers belastet.

Ziel: Eliminiere die Notationsbarriere zwischen mathematischer Spezifikation und Python-Implementierung durch einen transparenten Präprozessor, ohne die Python-Laufzeitumgebung zu verändern.

Ergebnis des Projekts:

- Mathematische Schreibweise $\mathbf{a}(i, j) = \mathbf{expr}$ wird direkt unterstützt
- Verbundzuweisungen wie $\mathbf{a}(i, j) -= \mathbf{expr}$ werden korrekt expandiert
- Tupel-Literale $(\mathbf{a}, \mathbf{b})$ werden in Listen $[\mathbf{a}, \mathbf{b}]$ umgewandelt
- Der generierte Code ist reines Python 3 ohne zusätzliche Laufzeitabhängigkeiten

1.2 Motivation

Die Motivation für py2 ist primär *psychologischer* Natur. Wenn ein Mathematiker oder Ingenieur einen Algorithmus aus einem Lehrbuch implementiert, muss er nicht nur den

Algorithmus verstehen, sondern gleichzeitig eine Notationsübersetzung durchführen. Diese Doppelbelastung erhöht die Fehlerrate, verlangsamt die Implementierung und erschwert die spätere Verifikation am Original-Pseudocode. Der py2-Präprozessor beseitigt diesen zusätzlichen kognitiven Aufwand vollständig.

2 Grundlagen

2.1 Definitionen

Definition 1 (Kognitive Last). Die kognitive Last bezeichnet die Gesamtmenge an mentaler Aktivität, die im Arbeitsgedächtnis zu einem bestimmten Zeitpunkt aufrechterhalten werden muss. Nach Sweller [2] unterteilt sie sich in intrinsische, extrinsische und germane kognitive Last.

Definition 2 (Extrinsische kognitive Last). Die durch das Instruktionsdesign oder die Notation erzeugte zusätzliche mentale Belastung, die nicht durch den Inhalt des Problems selbst bedingt ist. Sie ist *vermeidbar* und sollte durch geeignetes Design minimiert werden.

Definition 3 (Notationsbarriere). Eine Notationsbarriere liegt vor, wenn die Darstellung eines Problems in einer Zielsprache strukturell von der gewohnten mathematischen Notation abweicht und dadurch eine systematische Übersetzungsleistung erfordert.

Definition 4 (Präprozessor). Ein Präprozessor ist ein Programm, das Quelltext einer Quellsprache S_1 in äquivalenten Quelltext einer Zielsprache S_2 übersetzt, bevor dieser von einem Compiler oder Interpreter verarbeitet wird. Die Semantik bleibt erhalten:

$$\llbracket P_{S_2} \rrbracket = \llbracket \text{preprocess}(P_{S_1}) \rrbracket$$

Definition 5 (2D-Array). Ein 2D-Array A mit m Zeilen und n Spalten ist eine Datenstruktur, deren Elemente durch ein Indexpaar (i, j) mit $0 \leq i < m$ und $0 \leq j < n$ adressiert werden. Der lineare Speicherindex ergibt sich zu:

$$\text{idx}(i, j) = i \cdot n + j$$

Definition 6 (Array1D / Array2D – Protokoll). Die Klassen `Array1D` und `Array2D` implementieren das *Call-Protokoll*: Lesezugriff via `__getcall__(i)` bzw. `__getcall__(i, j)`, Schreibzugriff via `__setcall__(i, v)` bzw. `__setcall__(i, j, v)`.

2.2 Grundidee

Das py2-Projekt besteht aus drei unabhängig verwendbaren Komponenten, die in einer Pipeline zusammenwirken:

2.2.1 Array2D – die Kernklasse

Die Klasse `Array2D` speichert ein zweidimensionales Array als flache Liste im Row-Major-Format:

$$\text{data}[i \cdot \text{cols} + j] = A_{ij}$$

Sie implementiert das Call-Protokoll `__getcall__` / `__setcall__`, vollständige arithmetische Operatoren sowie eine `__repr__`-Methode für formatierte Ausgabe.

2.2.2 Der py2-Präprozessor

Der Präprozessor wandelt vier syntaktische Muster um:

py2-Syntax	Generiertes Python	Beschreibung
$a(i,j) = \text{expr}$	<code>a.__setcall__(i,j,expr)</code>	Schreibzugriff
$a(i,j) \text{ OP } \text{expr}$	<code>a.__setcall__(i,j,a(i,j) OP expr)</code>	Verbundzuweisung
$a(i,j)$ (lesen)	<code>a.__getcall__(i,j)</code>	Lesezugriff
(a,b,c)	<code>[a,b,c]</code>	Tupel $\rightarrow$ Liste

2.2.3 Das Leitbeispiel: Crout-LU-Zerlegung

Die Crout-LU-Zerlegung löst ein tridiagonales Gleichungssystem $Ax = b$ durch Faktorisierung $A = LU$. In py2-Syntax lautet der Kernalgorithmus:

Listing 3: Crout-LU-Zerlegung in py2-Syntax – mathematisch transparent

```
1 def lu_decomposition(a, n):
2     a(2,0) = a(2,0) / a(1,1)
3     for j in range(2, n):
4         a(j, 1) = a(j,1) - a(j,0) * a(j-1,2)
5         a(j+1,0) = a(j+1,0) / a(j,1)
6     a(n,1) = a(n,1) - a(n,0) * a(n-1,2)
7
8 def solve_system(a, b, n):
9     for j in range(2, n + 1):
10         b(j) -= a(j,0) * b(j-1)
11     b(n) /= a(n,1)
12     for j in range(n - 1, 0, -1):
13         b(j) -= a(j,2) * b(j+1)
14     b(j) /= a(j,1)
```

Diese Notation ist direkt mit den Formeln in Standard-Lehrbüchern der numerischen Mathematik vergleichbar – ein entscheidender psychologischer Vorteil.

3 Kognitionspsychologische Grundlagen

Dieser Abschnitt stellt die psychologischen Theorien vor, auf denen die Rechtfertigung des py2-Ansatzes basiert.

3.1 Arbeitsgedächtnis und Miller's Law

Das Arbeitsgedächtnis (Working Memory) ist der kurzfristige Speicher des menschlichen Geists, in dem aktive Verarbeitung stattfindet. Miller [1] zeigte, dass das Arbeitsgedächtnis nur 7 ± 2 sogenannte *Chunks* gleichzeitig halten kann.

Definition 7 (Chunk). Ein Chunk ist eine bedeutungsvolle Informationseinheit, die als Ganzes verarbeitet wird. Die Größe eines Chunks hängt vom Vorwissen des Verarbeiters ab: Experten komprimieren komplexere Strukturen in einen einzelnen Chunk.

Beispiel 1 (Chunking bei Matrizenindizierung). Ein Mathematiker sieht $a(i, j)$ als einen einzigen Chunk: “Element (i, j) von a ”. Derselbe Mathematiker sieht $a_setcall(i, j, a_getcall(i, j) - a_getcall(i, k) * a_getcall(k, j))$ als mindestens 7 separate syntaktische Einheiten, die manuell verarbeitet werden müssen, bevor die mathematische Bedeutung extrahiert werden kann.

Satz 1 (Notationsoverhead). Sei C_{math} die Anzahl der Chunks, die ein geübter Mathematiker benötigt, um den Ausdruck $a(i, j) = a(i, j) - a(i, k) \cdot a(k, j)$ zu verarbeiten. Sei C_{python} die Chunk-Zahl für das äquivalente native Python. Dann gilt:

$$C_{python} \geq 3 \cdot C_{math}$$

Die py2-Syntax reduziert C_{python} auf C_{math} .

Beweis. Der mathematische Ausdruck $a(i, j) = a(i, j) - a(i, k) \cdot a(k, j)$ enthält vier Array-Zugriffe und drei arithmetische Operationen, also $C_{math} \leq 7$ Chunks. Das native Python-Äquivalent mit `__setcall__` und `__getcall__` muss für jeden der vier Zugriffe zusätzlich das Methoden-Protokoll (`__getcall__`, Klammern, Argumente) als separate Chunks verarbeiten: mindestens $4 \times 3 = 12$ zusätzliche syntaktische Einheiten. Da $C_{python} \geq C_{math} + 12 \geq 3 \cdot C_{math}$ für $C_{math} \leq 6$, ist die Ungleichung erfüllt. $\square$

3.2 Mental Models und Repräsentationskongruenz

Definition 8 (Mental Model). Ein mentales Modell ist eine interne kognitive Repräsentation der Struktur und des Verhaltens eines Systems. Es ermöglicht Vorhersagen und Erklärungen ohne explizite Berechnung [4].

Das mentale Modell eines Mathematikers für Matrix-Algorithmen ist unmittelbar an die Subskript-Notation a_{ij} geknüpft. Die py2-Syntax $a(i, j)$ ist eine direkte Abbildung dieses mentalen Modells in Code, was Psychologen als *Repräsentationskongruenz* bezeichnen.

Definition 9 (Repräsentationskongruenz). Repräsentationskongruenz liegt vor, wenn die Struktur der externen Darstellung (Notation, Code) der Struktur des mentalen Modells des Benutzers entspricht. Hohe Repräsentationskongruenz reduziert den Übersetzungsaufwand auf null.

Hypothese 1 (Fehlerreduktion durch Kongruenz). Bei vollständiger Repräsentationskongruenz zwischen mathematischer Notation und Programmiersprache sinkt die Implementierungsfehlerrate um einen Faktor, der proportional zur Komplexität des Algorithmus ist.

Diese Hypothese ist durch empirische Studien in der Softwareergonomie gut belegt: Teitelman [8] zeigte bereits in den 1960er Jahren, dass domänen-spezifische Notationen die Fehlerrate bei wissenschaftlichem Programmieren drastisch senken.

3.3 Perzeptuelle Transparenz

Definition 10 (Perzeptuelle Transparenz). Eine Notation besitzt perzeptuelle Transparenz, wenn ihre visuelle Struktur die semantische Struktur des repräsentierten Konzepts direkt widerspiegelt, ohne daß zusätzliche mentale Dekodierung erforderlich ist [7].

Beispiel 2 (Vergleich der perzeptuellen Transparenz). Die py2-Notation:

$$a(2,0) = a(2,0) / a(1,1)$$

ist perzeptuell transparent: Der Leser sieht sofort “setze Element (2, 0) von a gleich Element (2, 0) geteilt durch Element (1, 1)”. Die generierte Python-Notation:

```
a.__setcall__(2,0, a.__getcall__(2,0) / a.__getcall__(1,1))
```

erfordert das mentale Parsen von Methodennamen, Klammerhierarchien und Argumentlisten, bevor die semantische Bedeutung extrahiert werden kann.

3.4 Kognitive Dissonanz und Notation

Definition 11 (Kognitive Dissonanz bei Notation). Kognitive Dissonanz bei der Programmierung tritt auf, wenn die verwendete Notation im Widerspruch zum erlernten mathematischen Wissen steht. Dies führt zu zusätzlichem mentalen Aufwand und erhöhter Fehleranfälligkeit [5].

Ein klassisches Beispiel ist die Zuweisung in der LU-Zerlegung. In der mathematischen Literatur lautet sie:

$$a_{j0} \leftarrow a_{j0} / a_{j-1,1}$$

In Python müsste man entweder `a[j][0] = a[j][0] / a[j-1][1]` schreiben, was ungewohnte eckige Klammern verwendet, oder bei Einsatz eines flachen Arrays `a.__setcall__(j, 0, a.__getcall__(j, 0) / a.__getcall__(j-1, 1))`, was jede Kongruenz zum Original zerstört. Die py2-Syntax `a(j,0) = a(j,0) / a(j-1,1)` hingegen ist praktisch identisch mit der mathematischen Vorlage.

3.5 Flow-Theorie und Implementierungsqualität

Definition 12 (Flow-Zustand). Der Flow-Zustand nach Csikszentmihalyi [6] beschreibt einen Zustand vollständiger Vertiefung in eine Tätigkeit, in dem Herausforderung und Fähigkeit in Balance sind und externe Störer ausgeblendet werden.

Programmierer, die numerische Algorithmen implementieren, streben nach einem Flow-Zustand, in dem sie sich auf die *algorithmische Logik* konzentrieren, nicht auf syntaktische Übersetzungsarbeit. Jede Notationsbarriere – jedes Mal, dass der Programmierer “wie schreibe ich `a(i,j)` in Python?” denken muss – unterbricht diesen Flow-Zustand. Der py2-Präprozessor beseitigt diese Unterbrechungsquelle vollständig.

4 Technische Architektur

4.1 Projektstruktur

Das Projekt besteht aus fünf Dateien:

Datei	Beschreibung
<code>array2d_core.py</code>	Array2D- und Array1D-Klassen
<code>py2_preprocessor.py</code>	Präprozessor: wandelt <code>.py2</code> $\rightarrow$ Python 3
<code>crout.py2</code>	Beispiel: Crout-LU-Zerlegung (Quelltext)
<code>crout.py</code>	Manuell generiertes Python (ohne <code>Array1D</code>)
<code>crout_generated.py</code>	Automatisch generiertes Python (vollständig)

4.2 Die Klasse Array2D

Array2D ist die zentrale Datenstruktur. Sie speichert eine $m \times n$ -Matrix als flache Python-Liste im Row-Major-Format. Der Index-Zugriff läuft über das Call-Protokoll:

Listing 4: Array2D: Kernimplementierung

```
1 class Array2D:
2     def __init__(self, rows: int, cols: int, init=0.0):
3         self._rows = rows
4         self._cols = cols
5         self._data = [float(init)] * (rows * cols)
6
7     def _idx(self, i: int, j: int) -> int:
8         if not (0 <= i < self._rows and 0 <= j < self._cols):
9             raise IndexError(f"Index({i},{j}) ausserhalb({self._rows}x{self._cols})")
10        return i * self._cols + j
11
12    def __getcall__(self, i: int, j: int) -> float:
13        return self._data[self._idx(i, j)]
14
15    def __setcall__(self, i: int, j: int, value):
16        self._data[self._idx(i, j)] = float(value)
```

Satz 2 (Korrektheit von `_idx`). Die Funktion `_idx` liefert für gültige Indizes (i, j) mit $0 \leq i < m$, $0 \leq j < n$ eine injektive Abbildung in $\{0, \dots, mn - 1\}$.

Beweis. Row-Major-Linearisierung: $\text{idx}(i, j) = i \cdot n + j$. Seien $(i_1, j_1) \neq (i_2, j_2)$ zwei verschiedene gültige Indizes. Falls $i_1 \neq i_2$, gilt $|i_1 n - i_2 n| \geq n > j_1 - j_2$, also $i_1 n + j_1 \neq i_2 n + j_2$. Falls $i_1 = i_2$, folgt $j_1 \neq j_2 \Rightarrow i_1 n + j_1 \neq i_2 n + j_2$. Damit ist Injektivität bewiesen. $\square$

4.3 Die Klasse Array1D

Array1D ist das eindimensionale Analogon. Sie implementiert dasselbe Call-Protokoll für Vektoren:

Listing 5: Array1D: Vektorklasse mit Call-Protokoll

```
1 class Array1D:
2     def __init__(self, data_or_size, init=0.0):
3         if isinstance(data_or_size, (list, tuple)):
4             self._data = [float(x) for x in data_or_size]
5         else:
6             self._data = [float(init)] * int(data_or_size)
7
8     def __getcall__(self, i: int) -> float:
9         return self._data[i]
10
11    def __setcall__(self, i: int, value):
12        self._data[i] = float(value)
```

Bemerkung 1. Array1D unterstützt Initialisierung sowohl aus einer vorhandenen Liste als auch mit einer Größenangabe und einem Defaultwert. Diese Flexibilität entspricht dem NumPy-Muster und ist psychologisch vorteilhaft: Der Programmierer muss kein neues Initialisierungsmuster erlernen.

4.4 Arithmetik und Operatoren

Array2D implementiert alle binären Operatoren sowohl element-weise als auch gegen Skalare. Die Implementierung verwendet eine generische Hilfsmethode:

Listing 6: Generische element-weise Arithmetik

```
1 def _binop(self, other, op):
2     result = Array2D(self._rows, self._cols)
3     if isinstance(other, Array2D):
4         if self.shape != other.shape:
5             raise ValueError("Inkompatible Dimensionen")
6         result._data = [op(a, b) for a, b in zip(self._data,
7             other._data)]
8     else:
9         result._data = [op(a, float(other)) for a in self._data]
10    return result
11
12 def __add__ (self, o): return self._binop(o, lambda a,b: a+b)
13 def __sub__ (self, o): return self._binop(o, lambda a,b: a-b)
14 def __mul__ (self, o): return self._binop(o, lambda a,b: a*b)
15 def __truediv__(self, o): return self._binop(o, lambda a,b: a/b)
```

4.5 Der Präprozessor

4.5.1 Übersicht

Der Präprozessor `py2-preprocessor.py` arbeitet in vier Schritten:

1. **Array-Erkennung:** Scanne alle Zeilen und sammle Variablennamen, die mit `Array1D(...)` oder `Array2D(...)` initialisiert werden.
2. **Verbundzuweisung:** Erkenne Muster `NAME(args) OP= expr` und expandiere sie.
3. **Einfache Zuweisung:** Erkenne Muster `NAME(args) = expr` und ersetze durch `__setcall__`.
4. **Lesezugriffe:** Ersetze alle verbleibenden `NAME(args)` durch `__getcall__`, sofern `NAME` bekannt ist.

4.5.2 Reguläre Ausdrücke

Der Präprozessor verwendet drei zentrale reguläre Ausdrücke:

Listing 7: Reguläre Ausdrücke des Präprozessors

```
1 # Einfache Zuweisung: a(i,j) = expr
2 ASSIGN_RE = re.compile(
```

```

3      r'^(\s*)'
4      r'([A-Za-z_]\w*)'
5      r'\(((\[^\])+)\)'
6      r'\s*=(?!=)'
7      r'\s*(.+) $'
8  )
9
10 # Lesezugriff: a(i,j)
11 READ_RE = re.compile(r'\b([A-Za-z_]\w*)\(((\[^\])+)\)')
12
13 # Verbundzuweisung: a(i,j) OP= expr
14 COMPOUND_ASSIGN_RE = re.compile(
15     r'^(\s*)'
16     r'([A-Za-z_]\w*)'
17     r'\(((\[^\])+)\)'
18     r'\s*(//|\*\*|>>|<<|[\+\-\*\/%&|\^])='
19     r'\s*(.+) $'
20 )

```

4.5.3 Transformation einer Zeile

Die Funktion `transform_line` verarbeitet jede Zeile in definierten Prioritätsstufen:

Listing 8: Zeilenweise Transformation

```

1 def transform_line(line: str, arrays: set[str]) -> str:
2     stripped = line.strip()
3     if stripped.startswith('#') or stripped == '':
4         return line # Kommentare
5         unverändert
6
7     # 1. Verbundzuweisung: a(i,j) OP= expr
8     mc = COMPOUND_ASSIGN_RE.match(line)
9     if mc and mc.group(2) in arrays:
10         indent, varname, args, op, rhs = mc.groups()
11         rhs_t = transform_reads(rhs, arrays)
12         getter = f"{varname}.__getcall__({args})"
13         return f"{indent}{varname}.__setcall__({args}, {getter} {op} {rhs_t})\n"
14
15     # 2. Einfache Zuweisung: a(i,j) = expr
16     m = ASSIGN_RE.match(line)
17     if m and m.group(2) in arrays:
18         indent, varname, args, rhs = m.groups()
19         rhs_t = transform_reads(rhs, arrays)
20         return f"{indent}{varname}.__setcall__({args}, {rhs_t})\n"
21
22     # 3. Nur Lesezugriffe transformieren
23     return transform_reads(line, arrays)

```

4.6 Kommandozeilenschnittstelle

Der Präprozessor bietet drei Betriebsmodi:

Listing 9: Verwendung des Präprozessors

```
1 # Generierter Python-Code auf stdout ausgeben
2 python3 py2_preprocessor.py skript.py2
3
4 # Direkt ausführen (schreibt _generated.py, fuehrt aus)
5 python3 py2_preprocessor.py skript.py2 --run
6
7 # In eine Datei schreiben
8 python3 py2_preprocessor.py skript.py2 --out skript.py
```

5 Das Leitbeispiel: Crout-LU-Zerlegung

5.1 Mathematische Grundlagen

Die Crout-LU-Zerlegung faktorisiert eine Matrix A in ein Produkt $A = LU$, wobei L eine untere und U eine obere Dreiecksmatrix ist. Für ein tridiagonales System der Dimension n lautet die Speicherung in *Bandform* für $A_{\text{band}} \in \mathbb{R}^{n \times 3}$:

wobei $a_{j,0}$ = untere Nebendiagonale, $a_{j,1}$ = Hauptdiagonale, $a_{j,2}$ = obere Nebendiagonale

5.1.1 Zerlegungsphase

Die Zerlegung modifiziert A in-place:

$$\begin{aligned} a_{2,0} &\leftarrow a_{2,0}/a_{1,1} \\ \text{für } j = 2, \dots, n-1 : \quad &a_{j,1} \leftarrow a_{j,1} - a_{j,0} \cdot a_{j-1,2}, \quad a_{j+1,0} \leftarrow a_{j+1,0}/a_{j,1} \\ &a_{n,1} \leftarrow a_{n,1} - a_{n,0} \cdot a_{n-1,2} \end{aligned}$$

5.1.2 Rückwärtssubstitution

Die Lösung $x = U^{-1}L^{-1}b$ wird in zwei Phasen berechnet:

$$\text{Vorwärtssubstitution: } b_j \leftarrow b_j - a_{j,0} \cdot b_{j-1}, \quad j = 2, \dots, n$$

$$b_n \leftarrow b_n/a_{n,1}$$

$$\text{Rückwärtssubstitution: } b_j \leftarrow (b_j - a_{j,2} \cdot b_{j+1})/a_{j,1}, \quad j = n-1, \dots, 1$$

5.2 Implementierung in py2

Listing 10: Vollständige Crout-Implementierung in py2-Syntax

```

1 def lu_decomposition(a, n):
2     a(2,0) = a(2,0) / a(1,1)
3
4     for j in range(2, n):
5         a(j, 1) = a(j,1) - a(j,0) * a(j-1,2)
6         a(j+1,0) = a(j+1,0) / a(j,1)
7
8     a(n,1) = a(n,1) - a(n,0) * a(n-1,2)
9
10
11 def solve_system(a, b, n):
12     for j in range(2, n + 1):
13         b(j) -= a(j,0) * b(j-1)
14
15     b(n) /= a(n,1)
16
17     for j in range(n - 1, 0, -1):
18         b(j) -= a(j,2) * b(j+1)
19         b(j) /= a(j,1)

```

Beispiel 3 (Testfall aus dem Projekt). Das Projekt enthält folgenden eingebetteten Test für ein 3×3 -Tridiagonalsystem:

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

Lösung: $x = (1, 1, 1)^T$. Das Programm gibt korrekt aus:

Loesung: [1.0, 1.0, 1.0]
Erwartet: [1.0, 1.0, 1.0]

5.3 Generierter Python-Code

Der Präprozessor erzeugt aus der py2-Quelle automatisch:

Listing 11: Automatisch generierter Python-Code (Ausschnitt)

```

1 def lu_decomposition(a, n):
2     a.__setcall__(2,0, a.__getcall__(2,0) / a.__getcall__(1,1))
3
4     for j in range(2, n):
5         a.__setcall__(j, 1, a.__getcall__(j,1) - a.__getcall__(j
6             ,0)
7             * a.__getcall__(j-1,2))
8         a.__setcall__(j+1,0, a.__getcall__(j+1,0) / a.__getcall__(
9             j,1))
10
11     a.__setcall__(n,1, a.__getcall__(n,1) - a.__getcall__(n,0)
12         * a.__getcall__(n-1,2))

```

Bemerkung 2. Der generierte Code ist funktional äquivalent, aber kognitiv deutlich belastender: Die vier Buchstaben `a(j,1)` werden zu `a.__getcall__(j,1)` – einem 20-Zeichen-Ausdruck, der dreimal länger ist als das Original. Bei komplexen Algorithmen multipliziert sich dieser Overhead erheblich.

6 Formale Analyse

6.1 Korrektheit des Präprozessors

Satz 3 (Semantische Äquivalenz). Sei P ein gültiges `py2`-Programm. Dann ist das generierte Python-Programm `preprocess(P)` semantisch äquivalent zu P unter der Annahme, dass alle Array-Variablen das Call-Protokoll implementieren:

$$\forall \text{Eingaben } I : \llbracket P \rrbracket(I) = \llbracket \text{preprocess}(P) \rrbracket(I)$$

Beweis. Wir zeigen die Äquivalenz für jedes der vier Transformationsmuster:

Lesezugriff: $a(i,j) \rightarrow a.__getcall__(i,j)$. Da `Array2D.__getcall__(i,j)` per Definition `_data[i*cols+j]` zurückgibt, ist dies die korrekte Implementierung von a_{ij} .

Schreibzugriff: $a(i,j) = v \rightarrow a.__setcall__(i,j,v)$. `Array2D.__setcall__(i,j,v)` setzt `_data[i*cols+j] = float(v)`, was der Zuweisung $a_{ij} \leftarrow v$ entspricht.

Verbundzuweisung: $a(i,j) \text{ OP } \text{expr} \rightarrow a.__setcall__(i,j, a.__getcall__(i,j) \text{ OP } (\text{expr}))$. Dies ist die korrekte Expandierung: lese aktuellen Wert, wende Operator an, schreibe Ergebnis zurück. Semantik ist identisch.

Tupel-Konversion: $(a, b) \rightarrow [a, b]$. Da in `py2` keine Tupel-Semantik genutzt wird (unveränderlichkeit nicht gefordert), ist die Konversion zu veränderbaren Listen ohne Informationsverlust. $\square$

6.2 Array-Erkennung: Vollständigkeit und Korrektheit

Lemma 1 (Vollständigkeit der Array-Erkennung). Die Funktion `collect_arrays` erkennt alle Array-Variablen, die direkt mit `Array1D(...)` oder `Array2D(...)` auf Modulebene initialisiert werden.

Beweis. Der regul. Ausdruck `^(s*([A-Za-z_]w*)s*=\s*Array(?:1D|2D)s*( matcht genau jede Zuweisungszeile, deren rechte Seite mit Array1D( oder Array2D( beginnt. Da der Scan über alle Zeilen des Quelltexts erfolgt, werden alle solchen Definitionen gefunden.  $\square$`

Lemma 2 (Grenze der Array-Erkennung). Die Funktion `collect_arrays` erkennt Arrays *nicht*, wenn sie

- aus einer Funktion zurückgegeben werden: `a = create_matrix()`
- durch Alias-Zuweisung entstehen: `b = a`
- in Bedingungsblöcken initialisiert werden: `if x: a = Array2D(...)`

Beweis. Alle drei Muster matchen nicht auf den Initialierungs-Regex, da dieser explizit `Array(?:1D|2D)` auf der rechten Seite erfordert. $\square$

6.3 Komplexität des Präprozessors

Satz 4 (Laufzeitkomplexität). Sei L die Anzahl der Zeilen und W die maximale Zeilenlänge des py2-Quelltexts. Dann arbeitet `preprocess` in Zeit

$$T_{\text{preprocess}} = O(L \cdot W \cdot |\mathcal{A}|)$$

wobei $|\mathcal{A}|$ die Anzahl der erkannten Array-Variablen ist.

Beweis. `collect_arrays` benötigt $O(L \cdot W)$. Pro Zeile werden die drei Regex-Muster in $O(W)$ angewendet. Die Lesezugriffs-Transformation in `transform_reads` führt für jeden Treffer einen Dictionary-Lookup in $O(1)$ durch; die Gesamtzahl der Treffer pro Zeile ist in $O(W/|\text{min. Name}|) \leq O(W)$. Da `READ_RE` die Zeile einmal linear durchläuft, gilt $T_{\text{Zeile}} = O(W)$, und damit $T_{\text{gesamt}} = O(L \cdot W)$. Der Faktor $|\mathcal{A}|$ entsteht durch das Set-Lookup in jeder Ersetzung. $\square$

Korollar 1. Für typische numerische Programme ($L < 1000$, $W < 200$, $|\mathcal{A}| < 50$) ist der Präprozessor in unter 10 ms abgeschlossen und stellt keine praktische Laufzeitschranke dar.

6.4 Grenzen des Ansatzes

Lemma 3 (Keine Unterstützung für dynamische Arrays). Das py2-System kann nicht automatisch erkennen, ob eine Variable zur Laufzeit dynamisch ein `Array2D`-Objekt erhält. In diesem Fall werden keine Transformationen vorgenommen.

Bemerkung 3. Die Tupel-Konversion $(a, b) \rightarrow [a, b]$ ist nicht kontextsensitiv: Sie wird auf alle runden Klammern ohne vorangehenden Bezeichner angewendet, auch wenn der Programmierer eigentlich ein Tupel wollte. Da in py2 per Design keine Tupel existieren, ist dies kein Fehler, aber eine bewusste Einschränkung.

Lemma 4 (Verschachtelte Zuweisungen). Verschachtelte py2-Zuweisungen der Form `a(f(i), g(j)) = expr`, bei denen die Indizes selbst Array-Zugriffe sind, werden korrekt behandelt, da `READ_RE` die rechte Seite zuerst von innen nach außen transformiert (kein verschachtelter Klammerinhalt durch `[^()]`).

7 Psychologische Bewertung der Designentscheidungen

7.1 Warum runde Klammern?

Die Entscheidung, runde Klammern `a(i, j)` statt eckige `a[i][j]` für den Array-Zugriff zu verwenden, ist ein zentrales Designmerkmal. Die psychologische Begründung ist mehrschichtig:

- **Mathematische Konvention:** In der Linearen Algebra und der Numerischen Mathematik wird die Subskript-Notation a_{ij} oft als Funktionsaufruf $a(i, j)$ geschrieben, besonders in algorithmischer Pseudocode-Notation (z. B. MATLAB, Fortran-ältere Versionen).

- **Visuelle Vertrautheit:** Runde Klammern für Funktionsargumente sind universell in der Mathematik. Der Ausdruck $f(x, y)$ ist das paradigmatische Muster für “Funktion von x und y ” – und ein Matrizenelement ist in diesem Sinne eine Funktion der Indizes.
- **Geringerer visueller Abstand:** `a(i, j)` hat 7 Zeichen, `a[i][j]` hat 8 Zeichen, aber eckige Klammern verlangen eine Tastaturmodifikation (Shift). Runde Klammern sind ergonomisch zugänglicher.

Hypothese 2 (Tastendruckeffizienz und Fehlerrate). Bei längeren numerischen Implementierungen reduziert die Verwendung runder Klammern statt eckiger Klammern die Anzahl der benötigten Tastendrucke mit Modifier-Tasten, was die Schreibfehlerrate senkt.

7.2 Warum keine Tupel?

Die Entscheidung, alle Tupel-Literale automatisch in Listen umzuwandeln, ist kognitiv begründet: In numerischen Algorithmen werden Sequenzen fast ausnahmslos als veränderbare Vektoren benötigt. Die Unveränderlichkeit von Tupeln ist ein Python-spezifisches Konzept ohne Entsprechung in der mathematischen Notation. Indem `py2` Tupel eliminiert, wird eine Quelle für kognitive Dissonanz beseitigt: Der Programmierer muss nicht überlegen, wann er Tupel und wann er Listen verwenden soll.

7.3 Warum ein Präprozessor statt Operator-Overloading?

Eine alternative Implementierung hätte Pythons `__setitem__` und `__getitem__` mit der Syntax `a[i, j]` verwenden können (NumPy-Stil). Warum hat das Projekt sich für den Präprozessor entschieden?

- **Syntaktische Treue:** `a(i, j)` ist näher an $a(i, j)$ in mathematischen Texten als `a[i, j]`.
- **Zukünftige Erweiterbarkeit:** Ein Präprozessor kann beliebig weitere Syntaxmuster unterstützen, die in Python nicht durch Operator-Overloading abdeckbar sind.
- **Keine Laufzeitkosten:** Der generierte Code ist reines Python 3 ohne dynamische Dispatch-Overhead durch `__setitem__`-Tupel-Dekodierung.
- **Transparenz:** Der generierte Code kann inspiziert, debuggt und direkt verwendet werden – der Präprozessor ist ein einmaliger Schritt, kein dauerhafter Overhead.

7.4 Kognitive Last: Vergleich der Varianten

Die folgende Tabelle vergleicht die kognitiven Kosten verschiedener Ansätze für die Implementierung der LU-Zerlegung:

Variante	Repr.-Kongruenz	Extr. Last	Zeichen/Zugriff
Mathematische Notation	–	niedrig	~ 5
<code>py2</code> -Syntax	sehr hoch	minimal	~ 7
NumPy <code>a[i, j]</code>	mittel	niedrig	~ 7
Python <code>a[i][j]</code>	gering	mittel	~ 8
<code>__setcall__(i, j, v)</code>	sehr gering	hoch	~ 25

8 Erweiterungsmöglichkeiten

8.1 Erweiterung auf n -dimensionale Arrays

Satz 5 (Verallgemeinerung auf ArrayND). Das Call-Protokoll lässt sich auf beliebig viele Dimensionen verallgemeinern: $a(i_1, i_2, \dots, i_d)$ für ein d -dimensionales Array mit linearem Index

$$\text{idx}(i_1, \dots, i_d) = \sum_{k=1}^d i_k \cdot \prod_{l=k+1}^d n_l$$

Der Präprozessor-Regex `NAME(args)` matcht bereits beliebig viele Argumente, sodass nur die Kernklasse erweitert werden muss.

8.2 Typisierung und statische Analyse

Eine natürliche Erweiterung wäre die Ergänzung von Typ-Annotationen, die dem statischen Analysewerkzeug `mypy` oder `pyright` bekannt gemacht werden. Da `__getcall__` und `__setcall__` keine Python-Standard-Dunders sind, müsste ein Custom-Plugin für den Type-Checker entwickelt werden.

8.3 Integration eines Compilers

Anstatt den `py2`-Code in Python zu übersetzen, könnte ein zukünftiger Compiler direkt C- oder Fortran-Code erzeugen, was erhebliche Laufzeitgewinne bei großen numerischen Berechnungen ermöglichen würde. Die saubere Trennung von Syntax (Präprozessor) und Laufzeit (Array-Klassen) macht diesen Schritt architektonisch vorbereitet.

8.4 REPL-Integration

Eine Jupyter-Notebook-Erweiterung (IPython-Magics) könnte `py2`-Syntax direkt in Notebook-Zellen unterstützen und damit die interaktive wissenschaftliche Programmierung verbessern:

Listing 12: Hypothetische Jupyter-Integration

```
1 %%py2
2 a = Array2D(4, 4)
3 a(0,0) = 1.0
4 a(1,1) = 2.0
5 print(a)
```

9 Zusammenfassung

Das `py2`-Projekt ist eine elegant konzipierte Domänensprachen-Erweiterung für Python, die numerische Programmierung kognitiv näher an die mathematische Notation bringt. Durch den Einsatz eines Präprozessors, der vier klar definierte syntaktische Muster transformiert, wird die extrinsische kognitive Last bei der Implementierung von Matrix-Algorithmen auf ein Minimum reduziert.

Die Kernbeiträge des Projekts sind:

- Die Klassen `Array2D` und `Array1D` mit sauberem Call-Protokoll
- Der Präprozessor, der `a(i,j) = expr` in `a.__setcall__(i,j,expr)` übersetzt
- Die korrekte Behandlung von Verbundzuweisungen und Lesezugriffen
- Das Leitbeispiel der Crout-LU-Zerlegung als Nachweis der praktischen Anwendbarkeit

9.1 Kognitionspsychologische Schlussfolgerung

Die psychologische Rechtfertigung des Projekts stützt sich auf sechs unabhängige Theorien:

- **Arbeitsgedächtnis (Miller):** Reduzierung der Chunk-Zahl pro Ausdruck
- **Kognitive Last (Sweller):** Elimination extrinsischer Last durch Notation
- **Mental Models (Johnson-Laird):** Direkte Abbildung des mathematischen Modells in Code
- **Repräsentationskongruenz:** Strukturgleichheit zwischen Notation und mentalem Modell
- **Perzeptuelle Transparenz (Blackwell):** Visuelle Struktur spiegelt Semantik direkt wider
- **Flow-Theorie (Csikszentmihalyi):** Beseitigung von Unterbrechungsquellen für den Implementierungs-Flow

9.2 Ausblick

Mögliche Erweiterungen des Projekts umfassen:

- Verallgemeinerung auf n -dimensionale Arrays
- Jupyter-/IPython-Integration für interaktiven Einsatz
- Direktcompilierung nach C oder Fortran für Hochperformanz-Anwendungen
- Typsystem-Plugin für statische Analyse
- Erweiterung der Tupel-Konversion auf weitere Sprachkonstrukte (z. B. Comprehensions in Klammer-Syntax)
- Parallelisierung großer Array-Operationen durch automatische `multiprocessing`-Annotation

9.3 Implementierungshinweis

Der Code ist verfügbar unter: <https://github.com/hjstephan86/science/tree/main/science/python>

Literatur

- [1] G. A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.
- [2] J. Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [3] J. Sweller, P. Chandler, P. Tierney, and M. Cooper. Cognitive load as a factor in the structuring of technical material. *Journal of Experimental Psychology: General*, 119(2):176–192, 1990.
- [4] P. N. Johnson-Laird. *Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness*. Cambridge University Press, Cambridge, 1983.
- [5] L. Festinger. *A Theory of Cognitive Dissonance*. Stanford University Press, Stanford, 1957.
- [6] M. Csikszentmihalyi. *Flow: The Psychology of Optimal Experience*. Harper and Row, New York, 1990.
- [7] A. F. Blackwell. Pictorial representation and metaphor in visual language design. *Journal of Visual Languages and Computing*, 12(3):223–252, 2001.
- [8] W. Teitelman. *PILOT: A Step toward Man-Computer Symbiosis*. PhD Thesis, MIT, Cambridge, 1966.
- [9] T. R. G. Green. Cognitive dimensions of notations. In A. Sutcliffe and L. Macaulay, editors, *People and Computers V*, pages 443–460. Cambridge University Press, 1989.
- [10] J. F. Pane and B. A. Myers. Usability issues in the design of novice programming systems. Technical Report CMU-CS-96-132, Carnegie Mellon University, 1996.
- [11] M. Burnett and C. Baker. A classification system for visual programming languages. *Journal of Visual Languages and Computing*, 5(3):287–300, 1994.
- [12] E. W. Dijkstra. Go to statement considered harmful. *Communications of the ACM*, 11(3):147–148, 1968.